CHEAT SHEET · JS INTERVIEW #19 @specsycoder

What async returns

Why you get a Promise, not 5

async.js

```
async function getNumber() {
  return 5;
}
console.log(getNumber()); // Promise {5}
```

output **Promise {5}**

Why it happens

01 Always a promise
An async function ALWAYS returns a promise.

02 Values get wrapped
return 5 becomes Promise.resolve(5).

03 Errors get wrapped
throw inside async becomes a rejected promise.

04 Unwrap it
Use await or .then() to get the value.

How to get the value

get-value.js

```
const n = await getNumber(); // 5
getNumber().then((n) => console.log(n)); // 5
```

Save this · share with a friend Follow @specsycoder for daily JS interview Qs

async traps

What interviewers ask next

code	result
`async () => 5`	Promise {5}
`await outside async`	SyntaxError (except module top level)
`arr.forEach(async ...)`	does NOT wait
`for (const x of arr) await f(x)`	runs one by one
`await Promise.all(arr.map(f))`	runs in parallel

Traps to mention

- **Missing await** is a common bug: you log a Promise instead of the data.
- **try/catch works** with await to handle rejected promises.

Your 20-second answer

"An async function always returns a promise. If I return 5, JavaScript wraps it, so I get a promise that resolves to 5. To read the value I use await inside another async function, or .then(). Errors thrown inside become rejected promises.